
EVOKE: A KV-Cache Memory Hierarchy with Recompute-Free Block Recovery for LLM Serving

Anish Shrestha

Independent Researcher

siranishshrestha@gmail.com

github.com/Anyesh/EVOKE

Abstract

Long-running LLM agent sessions outgrow the physical KV cache budget within a few turns. Thinking models burn one to three thousand tokens of intermediate reasoning per turn, file-reading agents accumulate observations linearly, and once the cache overflows existing serving paths either truncate old history or re-prefill the entire conversation on every API call: the first loses information, the second pays full compute on every turn regardless of whether the older context turns out to matter. We present EVOKE (**EV**ict and **recO**ver **KV** cache **En**tries), an identity-keyed K/V recovery layer for the unified attention cache. EVOKE evicts low-relevance blocks via cheap position-metadata updates and recovers them recompute-free by splicing the original K and V tensors back into the cache at a new logical position with one RoPE phase shift and no forward pass. Recovery is identity-keyed: a recovery call names a specific block and reinstates the model’s original K and V tensors verbatim. A policy may use embedding similarity to score which evicted blocks are worth bringing back, but the recovered content is never the output of a re-encoding pass.

The system contribution is three new C primitives in a forked llama.cpp (`kv_block_save`, `kv_block_load`, `attn_capture`), a Python policy layer that scores blocks via a retrieval-tuned embedding model plus the model’s own attention, and an OpenAI-compatible chat-completions server that exposes EVOKE as a stateful endpoint with prefix-aware cache reuse. EVOKE is verified across three architectures: pure-attention Qwen 2.5 7B, hybrid Mamba/Attention Qwen 3.5 9B, and MoE Qwen 3.6 35B-A3B. On the standard needle-in-a-haystack benchmark at a $4\times$ compression ratio, EVOKE recovers 96 to 100% of needles across 25 (needle, depth) cells per architecture; recovery-less baselines (recency, StreamingLLM, `evoke_discard`, `evoke_breadcrumb`, H2O (Zhang et al., 2023), SnapKV (Li et al., 2024)) flatten to 20–40% at every budget, and a same-substrate InfLLM (Xiao et al., 2024b) adaptation reaches 96% at the tighter budgets but degrades to 80% at the loosest because its fixed aggressive footprint evicts the naturally-resident needle when budget headroom is plentiful (Section 7.5). On an agentic planted-fact head-to-head at a 1024-token budget (mean of $n=5$ runs), EVOKE matches the `no_eviction` baseline’s recall at $4\times$ smaller cache footprint and at truncate-comparable wall-clock (21.80s [21.55, 22.05] vs truncate’s 22.64s [19.81, 25.47] and `no_eviction`’s 19.62s [18.93, 20.31]), with the missing fact restored via 24 selective `kv_block_load` splices instead of per-turn re-prefill. The recovery primitive itself runs 20 to $32\times$ faster than equivalent re-prefill across block sizes from 20 to 1280 tokens (Section 7.1); end-to-end, factoring in the off-GPU save cost on the eviction path, the full save+load lifecycle is 5.9 to $7.5\times$ faster than re-prefill. Per-turn recovery cost is bounded by a smart-recovery policy that retrieves at most four blocks per user message via retrieval-grade embedding similarity,

gated against the best resident match so recoveries cannot pollute the cache with off-topic content.

1 Introduction

Production LLM serving handles cache overflow in one of two ways. The server either truncates the oldest history in the spirit of StreamingLLM, or it re-prefills the full conversation at every API call, which is what the OpenAI chat-completions default plus prefix caching amounts to. Truncation discards information that may be needed later. Re-prefill is expensive and pays the cost on every turn regardless of whether the prior context turned out to matter. Neither approach has any model of what content the agent will actually need next.

This paper argues that the right abstraction is the KV cache as a memory hierarchy with a recompute-free recovery primitive (Figure 1). Hot tokens stay in the active cache. Cold blocks are evicted to host RAM at metadata cost. When a future turn needs an evicted block, the block is spliced back into the active cache via a direct K and V tensor write, RoPE-rotated to its new logical position, with no forward pass. Prefix caching only avoids re-decoding an unchanged prefix and does nothing once the cache exceeds budget; Section 4 draws the precise line between EVOKE’s identity-keyed recovery and similarity-retrieved re-encoding (RAG).

The contributions of this paper are three:

1. **Three new C primitives in a forked llama.cpp.** `llama_kv_block_save` and `llama_kv_block_load` serialize a position range’s K/V tensors to a host buffer and splice them back into the unified cache with per-cell RoPE re-anchoring (Section 3). `llama_attn_capture_*` taps per-head softmax attention weights from one or more transformer layers into a host buffer once per decode (Section 3.3). Together they enable recompute-free identity-keyed K/V recovery in the unified attention cache and attention-aware eviction scoring; the fork is also extended to support hybrid Mamba+Attention memory and mrope position shifts, so the same primitives work across pure attention (Qwen 2.5), hybrid Mamba (Qwen 3.5), and MoE-with-thinking (Qwen 3.6 35B-A3B) (Section 6).
2. **A Python policy layer and OpenAI-compatible server** that turn the primitives into a usable system. The eviction scorer combines retrieval-tuned embedding coherence (`bge-small-en-v1.5`), the model’s own attention mass, recency, and harness-supplied priority. Recovery is routed through three pluggable backends (discard, breadcrumb, `kv_restore`) and gated by two policy decisions that proved load-bearing on retrieval-style workloads: smart-recovery scores breadcrumbs against the raw user-message text before the message is decoded so recovered blocks land as earlier context rather than the model’s freshest attention slot, and a resident-gate excludes any breadcrumb whose similarity does not beat the best non-current-turn resident block (Section 4, Section 5). The server reuses prefix-matched KV across turns and pools per-session KV state so a single model in VRAM can host many concurrent agent sessions.
3. **A head-to-head evaluation against seven baselines across three model families.** On the needle-in-a-haystack benchmark at $4\times$ compression, EVOKE recovers 96–100% of needles across 25 (needle, depth) cells per architecture, while every recovery-less baseline (recency, StreamingLLM, EVOKE’s own discard and breadcrumb backends, H2O (Zhang et al., 2023), and SnapKV (Li et al., 2024), all reimplemented in the same harness) flattens at 20–40%, and a same-substrate InfLLM (Xiao et al., 2024b) adaptation matches EVOKE within 4 points at tighter budgets but degrades at the loosest. On a 14-turn planted-fact agentic head-to-head ($n=5$, budget 1024), EVOKE matches the unconstrained `no_eviction` baseline’s recall at $4\times$ smaller cache footprint and at truncate-comparable wall-clock, while every recovery-less strategy fails the probe at every budget. Across block sizes from 20 to 1280 tokens, the `kv_block_load` primitive runs in 0.5 to 7 ms while the equivalent re-prefill takes 12 to 232 ms (20 to $32\times$ speedup, gap widens linearly with block size; 5.9 to $7.5\times$ over the full save+load lifecycle) (Section 7).

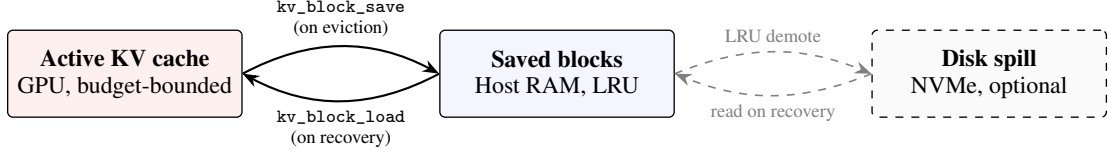


Figure 1: EVOKE’s memory hierarchy. The active KV cache holds the hot working set on GPU within a configured budget. On eviction, a block’s K and V tensors are serialized to host RAM via `kv_block_save`; recovery reverses the operation with `kv_block_load`, splicing the original tensors back into the active cache at a new logical position with one RoPE phase shift and no forward pass. An optional disk spill tier extends host RAM at NVMe latency.

2 Related Work

StreamingLLM (Xiao et al., 2024a) observed that the first few tokens of a sequence carry disproportionate attention mass and called them “attention sinks”. Their proposed serving policy keeps a small set of sink tokens at the start of the cache plus a sliding window of recent tokens, and drops everything in between. This works well for tasks that only need recent context but fails on tasks that need to recall earlier information. EVOKE keeps the sink-protection idea (sink-tagged blocks score 1.0 unconditionally in our scorer) but does not discard the middle: blocks evicted from the active cache are recoverable via the `kv_restore` backend.

H2O (Zhang et al., 2023) and **Scissorhands** (Liu et al., 2023) score KV entries by accumulated attention weight and drop the lowest-scoring entries to compress the cache. Both treat eviction as a one-way operation: dropped K/V is gone. EVOKE adopts the same attention-weight signal in its multi-signal scorer (Section 4) and pairs it with a recovery path so an eviction that turns out to be wrong is recoverable rather than terminal. We reimplement H2O’s heavy-hitter selection rule (cumulative attention with no decay, 10% recent-tail guard, hard-budget eviction) atop EVOKE’s `AttentionScorer` infrastructure for a same-substrate baseline; the head-to-head comparison is in Section 7.5.

SnapKV (Li et al., 2024) and **Ada-KV** (Feng et al., 2024) subsequently critiqued the H2O / Scissorhands family for biasing against recent tokens (cumulative scoring under-counts tail positions, which have received attention from fewer queries) and proposed batched single-pass selection and head-aware budgets respectively. EVOKE sidesteps the recent-bias issue by combining attention with recency as separate weighted signals, and the recent-tail guard imported for the H2O baseline is the same protected window SnapKV recommends. SnapKV is benchmarked head-to-head as a same-substrate baseline in Sections 7.4 to 7.6; the implementation freezes a one-shot snapshot of attention from the last 32 prompt tokens at the end of every user-message decode and keeps the top-scoring blocks for the rest of the turn (the paper’s default observation window).

LMCache (LMCache Team, 2024) and **DistAttention** (Lin et al., 2024) target cross-request KV reuse: LMCache stores prefix-matched KV bytes in a layered cache that can be shared across vLLM instances, and DistAttention disaggregates attention computation across a cluster so KV state need not co-locate with model weights. Both are below EVOKE’s policy layer: they handle “how the cached KV bytes move around the system”, while EVOKE handles “which blocks to evict from the active cache and how to splice them back in.” The two are composable; an EVOKE policy running on top of LMCache or a DistAttention serving fabric would inherit cross-request sharing for free.

vLLM and PagedAttention (Kwon et al., 2023) split the KV cache into fixed-size physical blocks indexed by a virtual page table, which is the same paging analogy this paper inherits at the OS level. **SGLang’s RadixAttention** (Zheng et al., 2024) extends this with a tree of cached prefixes shared across requests. Both systems are about how the cache is laid out and shared. EVOKE composes cleanly under both: the eviction and recovery primitives operate on logical position ranges and would apply equally well inside a paged allocator, though our implementation works directly on llama.cpp’s unified KV buffer.

RAG (Lewis et al., 2020) sidesteps the KV cache problem by storing text in a vector database and injecting retrieved passages back into the prompt. Retrieved tokens are re-encoded on every retrieval, and the model has no continuity with how those tokens were originally attended. EVOKE keeps the

original K and V tensors that the model computed at first sight of the tokens and splices them back at their original positions; Section 4 draws the precise line between identity-addressed recovery (which may still use similarity to choose *which* blocks to bring back) and similarity-retrieved re-encoding.

InfLLM (Xiao et al., 2024b) maintains a block-level external KV memory for long contexts and selects blocks back into the attention window via attention-score-based similarity. EVOKE shares the block-level memory-hierarchy framing but splices recovered blocks back into the unified contiguous KV cache (with RoPE re-anchored to the new logical position), where external-memory designs keep retrieved blocks in a separate memory segment that the attention layer reads alongside the active window. We benchmark InfLLM head-to-head as a same-substrate row in Sections 7.4 to 7.6: aggressive eviction down to a sinks-plus-local-window footprint, with per-turn top- $K=8$ smart recovery splicing the externally-held blocks back into the unified cache (a stand-in for InfLLM’s attention-mask redirection that preserves the block-level external-memory and similarity-retrieval choices).

Compressive transformers (Rae et al., 2019) and related infinite-memory architectures compress old KV into a smaller summary. EVOKE does not compress: it stores K/V exactly and recovers exactly. The tradeoff is that EVOKE needs host RAM proportional to evicted content, whereas compression bounds memory at the cost of fidelity.

RoPE re-anchoring is mechanically enabled by RoFormer’s rotary position embedding (Su et al., 2024). Because RoPE expresses position as a multiplicative phase on each K dimension pair, shifting a K row from one position to another reduces to applying a single rotation. llama.cpp already exposes this mechanism for its own internal seq_add shift. EVOKE reuses the same machinery on the recovery path: a saved K row written at a new position is correct after one queued shift.

Retrieval embeddings. Smart-recovery scoring in Section 4 uses BGE (Xiao et al., 2024c), specifically bge-small-en-v1.5, served through the fastembed runtime. The choice of a dedicated retrieval embedder rather than the LM’s intermediate hidden states is what makes block-vs-query similarity actually discriminative on retrieval-style workloads; Section 7.5 demonstrates the gap.

3 The C Primitives

The goal is to save a range $[p_0, p_1)$ of cells from sequence seq_id into a host buffer, then restore that buffer into the cache at a new starting position new_p0. Restoration must produce the same K and V tensors the model would attend to if positions new_p0 through new_p0 + $(p_1 - p_0)$ had been decoded in place.

The primitives are implemented in src/llama-context.cpp and exposed in include/llama.h:

```
size_t llama_kv_block_save(llama_context * ctx, llama_seq_id seq_id, llama_pos p0,
                           llama_pos p1, uint8_t * dst, size_t cap);
bool llama_kv_block_load(llama_context * ctx, llama_seq_id seq_id,
                        const uint8_t * src, size_t size, llama_pos new_p0);
```

llama_kv_block_save adapts the existing state_write_meta and state_write_data paths but filters to cells whose pos is in $[p_0, p_1)$ for seq_id. Per layer, we read the per-cell K row and V row via ggml_backend_tensor_get. We also store each cell’s original pos in the buffer because the deferred RoPE shift will need it.

llama_kv_block_load adapts the dest_seq_id != -1 path of state_read_data with one change: instead of seq_rm(seq, -1, -1), which would wipe the whole sequence, we clear only seq_rm(seq, new_p0, new_p0 + n_cells). We build a ubatch over $[new_p0, new_p0 + n_cells)$, call find_slot, apply_ubatch, and write the saved K and V via ggml_backend_tensor_set. We re-anchor RoPE by setting $shift[i] = new_pos - original_pos$ per cell, then run the existing K-shift graph via memory_update. V is never rotated.

This path runs no model forward pass: llama_decode is never called, the per-block cost is $O(n_cells \times n_layers \times n_embd_kv \times sizeof(dtype))$ of host-to-device memcpy plus a single K-rotation graph (the deferred RoPE shift llama.cpp already runs for its own seq_add path), and the V tensor is never touched after the initial save. Throughout this paper, “recompute-free” is shorthand

for “no model forward pass,” not literal zero compute; the latency numbers in Section 7.1 measure end-to-end load including the rotation.

3.1 Flash Attention is non-optional

With Flash Attention (Dao et al., 2022; Dao, 2023) disabled, V is stored transposed (`v_trans=true`). The `state_read_data` and `state_write_data` V loop then iterates `n_embd_v_gqa` times per layer. For Qwen 2.5 7B that is 28 layers times 512 `n_embd_v_gqa` per layer, which works out to 14,336 synchronous CUDA launches per load. With Flash Attention enabled (`v_trans=false`), V is row-aligned like K and the fast path runs 56 launches per load (two per layer, one for K and one for V). Measured impact on the eval host for a 20-token block: `kv_block_load` dropped from 133 milliseconds with FA disabled to 0.48 milliseconds with FA enabled. Flash Attention is therefore a hard requirement, not an optimization.

3.2 Cross-architecture extensions

The fork’s helper `llama_get_kv_cache`, which resolves a context’s `llama_memory_t` to an attention KV cache, is extended to unwrap two more memory types: `llama_memory_hybrid` (attention plus recurrent), via `get_mem_attn()`, and `llama_memory_hybrid_iswa` (attention with interleaved sliding-window attention plus recurrent), via `get_mem_attn()->get_base()`. Hybrid models carry a fixed-size recurrent state per layer that does not need eviction, so EVOKE operates on the attention component alone. The recurrent contribution to memory is preserved naturally as the model carries its blurry state forward.

For mrope models (Qwen 3.5 and later, anything with `n_pos_per_embd > 1`), the upstream `seq_add()` and `get_can_shift()` returned false outright. We removed those guards. `build_rope_shift` already has an mrope workaround that treats the temporal shift as a NeoX-style whole-vector rotation. Cells store a single temporal pos plus a separate ext payload for spatial axes, and shifting only the temporal pos is semantically correct for our position-only re-anchoring.

For hybrid Mamba plus Attention memories, `llama_memory_hybrid::seq_rm` dispatches to the recurrent half first. The recurrent half refuses partial tail rollback (a Mamba state cannot be sliced) and returns false, and the hybrid wrapper then returns false without mutating either sub-cache. A naive caller that interprets the return as success and updates its position bookkeeping will corrupt the cache on the next `llama_decode`; EVOKE’s Python `evict_ranges` now reads the return value and leaves bookkeeping intact on rejection.

For cases where the policy wants to drop attention cells without touching the recurrent state, two attention-only primitives, `llama_kv_block_seq_rm` and `llama_kv_block_seq_add`, are exposed in the fork. They reach the attention sub-cache directly via `llama_get_kv_cache` and bypass the hybrid wrapper, supporting a future no-shift eviction mode that the current policy layer does not yet exercise.

For iSWA models (Gemma 3 and 4, which interleave global and sliding-window attention across layers), the fork’s `llama_kv_block_save` and `llama_kv_block_load` save and restore both the base and SWA sub-caches via a newly added `llama_get_kv_cache_swa()` helper. The buffer layout for iSWA models is `[base block bytes][4-byte swa size][swa block bytes]`. For non-iSWA models the layout is unchanged. Python-side wiring to load a Gemma model and verify end-to-end recovery through EVOKE’s stack is not yet in place; only the fork primitives have been verified.

3.3 Attention-weight capture

The multi-signal scorer (Section 4) needs to know which past tokens the model is actively attending to as the conversation progresses. The strongest possible signal for this is the model’s own attention, the bilinear form $\text{softmax}(QK^\top/\sqrt{d})$ that determines the forward pass itself.

Flash Attention fuses the softmax inside its CUDA kernel and never materializes the attention weights as a tensor, and EVOKE requires Flash Attention to be on (Section 3.1: it dictates V ’s row-aligned layout, which is what makes the recovery primitives fast). Reading `kq_soft_max` from the existing graph is therefore not an option, and modifying the Flash Attention kernel itself to optionally emit

weights would mean a correctness-sensitive change across the CPU, CUDA, Metal, and Vulkan backends.

The lower-risk path is to compute a second softmax in parallel for one or more chosen layers. After Q and K are computed and permuted (the same Q and K that Flash Attention consumes), we add a separate graph node that computes $\text{softmax}(QK^\top, \text{scale} = \text{kq_scale}, \text{mask} = \text{kq_mask}, \text{sinks} = \text{sinks})$. The output is named `evoked_attn_capture` and force-expanded via `ggml_build_forward_expand` so the scheduler does not drop it as dead code. When `llama_decode` returns, `llama_context` copies the tensor data into a caller-owned host buffer via `ggml_backend_tensor_get`. The main attention path is unchanged: Flash Attention still runs, and the model output is byte-identical with capture off versus capture on.

The C API surface (added in `include/llama.h`):

```
LLAMA_API void llama_attn_capture_set_layer (llama_context *, int32_t layer);
LLAMA_API void llama_attn_capture_set_layers(llama_context *, const int32_t *
    layers, int32_t n);
LLAMA_API void llama_attn_capture_set_buffer(llama_context *, float * dst,
    size_t cap_floats);
LLAMA_API void llama_attn_capture_get_dims (const llama_context *, int32_t *
    n_layers,
    int32_t * n_q, int32_t * n_h,
    int32_t * n_kv);
LLAMA_API size_t llama_attn_capture_get_written(const llama_context *);
```

`set_layer` captures a single layer (legacy, kept bit-identical). `set_layers` accepts up to 16 layer indices and writes one slab per captured layer in a single decode. The buffer layout after each decode is `[n_layers, n_query, n_heads, n_kv]` in float32. For Qwen 2.5 7B at decode (`n_query=1`, `n_heads=28`, `n_kv` up to 16K), one layer slab is roughly 1.8 MB per step. Four layers fit in 7.2 MB, comfortably within the 32 MiB default buffer.

Validation. A real-model smoke test (`scripts/verify_attn_capture.py`) decodes a 31-token prompt on Qwen 2.5 7B with layer 20 tapped. It asserts that per-query softmax sums to 1.0, that values lie in $[0, 1]$, that the causal mask is respected (weights above the diagonal are exactly zero), and that the number of legal nonzero cells matches $\sum_i (i + 1) \cdot n_{\text{heads}}$ exactly. A second test (`scripts/verify_attn_capture_multi.py`) captures three layers at once (layers 4, 14, 20) in a single 29-token decode and asserts the same invariants per layer plus that distinct layers produce distinct attention patterns. Both pass on the eval host. The side-compute is numerically close but not bit-identical to the Flash Attention softmax, since the two paths run in different graph nodes; we verified this against non-FA mode for one test prompt and confirmed the difference is below the rounding tolerance of the f32 buffer.

For the scorer experiments in Sections 7.7 and 7.10, we use a single mid-layer (layer 20 for Qwen 2.5 7B), which we found sufficient to drive the relevance signal. Multi-layer aggregation is exposed in the API for future scorer designs.

3.4 Why eviction does not corrupt the cache

A common concern when a system mutates the KV cache mid-session is whether removing entries leaves dangling state. Working through a concrete example clarifies why EVOKE eviction is mathematically equivalent to “this token was never decoded,” not “this token is half-erased.”

Take a short sentence and assume one token per word for illustration. Treat the period as part of the preceding word (“color.”) so each shown token is one cell:

pos:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
tok:	Cat	sat	in	a	mat	and	mat	is	red	in	color.	house	is	green	in	color.	cat	is	very	pretty

That is 20 tokens at positions 0 through 19. Suppose the relevance scorer marks “house is green in color.” (positions 11 through 15, five tokens) as low-relevance. EVOKE evicts that range in two engine calls.

First, `seq_rm(seq=0, p0=11, p1=16)` walks the unified KV buffer and marks the five cells at those positions as free. K rows and V rows for that span are released. No dangling references exist because Q is never cached: it is recomputed fresh on every decode step and discarded. The cache only ever held K and V.

Second, `seq_add(seq=0, p0=16, p1=20, delta=-5)` does not move K or V tensors in memory. It updates the position metadata of the survivors “cat is very pretty” from positions 16 through 19 to 11 through 14, and queues a deferred RoPE shift of $\Delta = -5$ for those rows.

The K rows now need their RoPE phase corrected. llama.cpp applies the queued shift lazily at the next attention compute: each K row at new position p is multiplied by the rotation matrix $R(\Delta \cdot \theta_i)$ on every dimension pair, so its rotary angle becomes what it would have been if the token had been written at position 11 through 14 in the first place. V rows are untouched: V is positional-free, since RoPE only ever rotates K and Q .

A one-paragraph RoPE primer. Rotary Position Embedding rotates each consecutive dimension pair of K (and Q) by an angle $\theta_i(\text{pos}) = \text{pos} \cdot \theta_i$ where $\theta_i = \text{base}^{-2i/d}$ for dim pair i and head dim d (typically `base` = 10000). Because rotation is a multiplicative phase, the inner product $K(\text{pos}_k) \cdot Q(\text{pos}_q)$ collapses to a function of the relative position ($\text{pos}_k - \text{pos}_q$). That is the relative-position-encoding magic of RoPE. To re-anchor a K row from position 16 to position 11, you multiply it by $R(\Delta \cdot \theta_i)$ with $\Delta = -5$. The operation is exact, cheap (one rotation per dim pair), and requires no recompute of the underlying token.

After the shift, every surviving K vector is rotated exactly to match its new position. The attention math $Q_{\text{new}} \cdot K_{\text{survivor}}$ returns the same relative-position dot product it would have returned if “house is green in color.” had never been in the sequence. The model behaves identically to a model that decoded the truncated sentence directly. There is no torn attention state, no half-erased token, and no dangling reference. Eviction is a clean topological cut.

What can go wrong is not corruption but information loss. If the next user turn asks “what color was the house?”, the model has no K or V left for that fact and will either hallucinate or admit it does not know. That is precisely why recovery exists. `kv_restore` brings the saved K and V rows back, splices them in at a new contiguous position, and re-applies RoPE for the new anchor. The contract is “the cache contains exactly what you decided to keep; everything outside the cache is something you have to explicitly bring back.”

4 The EVOKE Policy Layer

The policy layer lives in `evoke/manager.py`, `evoke/scorer.py`, and `evoke/attention_scorer.py`. Each active block carries a score in $[0, 1]$ used by the watermark policy: when `active_tokens` crosses `high_watermark · budget`, the manager evicts the lowest-scoring blocks down to `low_watermark · budget`. The score combines four signals.

Attention (Section 3.3). Per-block softmax mass landing in that block over the last `n_window` decode steps, EWMA-weighted by decay. Defaults are a 64-step window and a decay of 0.95. This is the model’s own vote: blocks the recent query tokens actually attended to score high. When attention capture is unavailable (a stock llama.cpp build, or `w_attention=0`), this signal is absent and the score falls back to recency plus coherence.

Task-focus coherence. The scorer maintains a single task focus embedding rather than averaging the last N message embeddings. On each new user message the focus either updates via EMA (when the new message is coherent with the running focus, with cosine similarity to the focus $\geq \text{task_boundary_threshold}$, defaulting to 0.3 in the cosine sense, not as a weight), or it snaps to the new message. The snap is detected implicitly via the cosine drop, or signaled explicitly by the harness via `evoke_task_boundary=true`. Snapping makes blocks coherent with a prior task lose their coherence score in one scoring pass instead of decaying over five turns under the old rolling average.

Per-block embeddings can be drawn from two sources, switchable via `EvokeConfig.use_retrieval_embeddings`. The default mode pools the model’s own intermediate hidden states across the block tokens (`block_embedding_strategy="mean"`) —

cheap because the LM has already computed them, but the LM hidden state carries a strong common-mode component that collapses cosine similarities into a narrow 0.85–0.93 band across any two paragraphs in the same corpus, leaving no headroom for fine-grained retrieval. The opt-in retrieval mode embeds each block’s decoded text through a dedicated retrieval-tuned model (BAAI/bge-small-en-v1.5 via fastembed, ~30 MB, ~10 ms per text on CPU). The cosine band widens to 0.4–0.9 on the same workload, and the smart-recovery policy below (Section 5) can actually distinguish a needle block from haystack noise. Both block embeddings and the probe query embedding must come from the same space; the policy layer enforces this by routing both through the same embedder.

Recency. Exponential in the per-block distance from the current write position. A stability prior that prevents thrashing on a single high-attention spike.

Sink protection and source-type floors. Blocks marked `is_sink=True` (the first `sink_count` positions) score 1.0 unconditionally, which is the StreamingLLM-style attention anchor. USER and ASSISTANT turns get a score floor (defaults of 0.6 and 0.5 respectively) so that the conversation backbone is not evicted before document content.

The final score is $\text{raw} = (w_{\text{attn}} \cdot \text{attn} + w_{\text{rec}} \cdot \text{recency} + w_{\text{coh}} \cdot \text{coherence}) / \sum w$, lifted by source floor, multiplied by `block.priority`, and capped at 1.0. The `EvokeConfig` defaults are `w_attention=0.0`, `w_recency=0.4`, `w_coherence=0.6`: attention scoring is off out of the box, and a stock llama.cpp build that lacks the attention-capture primitive falls back to the recency-plus-coherence heuristic automatically. The Sections 7.7 and 7.10 attention experiments turn it on with `w_attention=0.5`, `w_recency=0.2`, `w_coherence=0.3`, which is the configuration we recommend for agent workloads where the eval host supports the EVOKE fork build. Note that the `w_coherence` weight (0.3 or 0.6 depending on configuration) and `task_boundary_threshold` (0.3 cosine) are unrelated despite the shared number; weights are linear combination coefficients while the threshold is a cosine cutoff for topic-shift detection.

Harness-supplied tags. The OpenAI chat-completions request gains three EVOKE-specific fields. `evoke_priority` is a float at least zero that multiplies the block’s final score. `evoke_pinned` is a boolean that excludes the block from eviction candidates entirely. `evoke_task_boundary` is a boolean that forces the coherence focus to snap. A harness like `opencode` or `Claude Code` that knows which file reads are central to the current task can set `priority=2.0` to keep them resident, and one-shot tool output can be marked `priority=0.3` so it is preferentially dropped under budget pressure. The fields default to 1.0, false, and false, so harnesses that ignore them get the model-and-heuristic behavior.

Pinned blocks are excluded from `_evictable_blocks` alongside sinks and the current-turn pin. `pin_generated=True` (the default) protects blocks created during the currently decoding turn so that generation does not immediately evict itself.

Recovery backends (`evoke/recovery.py`). Three pluggable strategies share a `RecoveryBackend` protocol.

- **DiscardBackend:** evicted content is gone. The agent must re-derive it.
- **BreadcrumbBackend:** a compact marker (key, token count, eviction step) is retained per evicted block. The agent (or harness) can choose to re-fetch the original text and call `add_context` to re-decode it.
- **KVRestoreBackend:** per-block K and V are saved to host RAM via `kv_block_save`. `recover(key)` looks up the saved buffer, calls `kv_block_load` at the next available position, and re-anchors RoPE. When a configurable RAM budget would otherwise drop an LRU victim, the backend can optionally spill the K and V bytes to disk under `kv_restore_spill_path`. Recovery then reads back from disk before splicing, at the cost of one NVMe read.

Block identity and the boundary with RAG. Every block carries a stable key such as "file:config.py#0", "user#42", or "assistant#43". Two properties distinguish EVOKE’s recovery from retrieval-augmented generation. First, addressability: a recovery call names a specific

key and the cache layer returns that exact block’s saved bytes, never a similarity-retrieved alternative. Second, content: what is spliced into the cache is the original K and V tensors the model computed when the tokens were first decoded, not text run back through a forward pass. The smart top-K policy in Section 5 *does* layer a small retrieval system over evicted-block embeddings to choose which keys to recover — that selection step is genuinely retrieval-shaped, and we note it here rather than dance around it. The boundary holds at the substitution step: similarity may choose the key, but the bytes spliced back are the K and V the model computed at first decode, never a re-encoding of nearby text.

5 The OpenAI-Compatible Server

The chat-completions API is stateless: every request resends the full message history, and the canonical server response is to re-prefill from scratch (or, with prefix caching, to reuse the unchanged prefix). EVOKE’s server (`evoke/server.py`, `evoke/session.py`) does both prefix caching and the eviction-and-recovery cycle under it.

Templating and prefix match. A persistent `Session` with one `EvokeManager` underneath outlives every request. On each incoming request the server templates the messages into raw text using the model’s own Jinja template parsed from the GGUF metadata: `llama_chat_apply_template` handles plain chat, and a Python `jinja2` path handles tool-using requests (the C API does not accept a tools array). The templated text is tokenized, then prefix-matched against the session’s `cached_tokens` to find the longest token prefix already in the physical KV cache.

Decode and eviction. Tokens past the prefix-match point are routed through `manager.add_context_tokens`, which decodes them, creates blocks, and invokes `_enforce_budget`. Eviction fires here under watermark pressure (Section 4).

Smart recovery. The session runs smart top-K recovery on each user turn (default $k = 4$), and three policy decisions in this loop are load-bearing for the head-to-head numbers in Sections 7.3 and 7.5.

First, scoring uses the retrieval-tuned embedder described in Section 4 when configured, with the query embedding derived from the raw user-message text rather than from an LM-hidden-state average over the probe’s KV positions. The LM hidden-state path collapses similarity into a narrow band where the needle block cannot beat haystack noise; the retrieval path widens that band by enough to make selection useful.

Second, recovery runs *before* the new user-message tail is decoded into the cache. In the original implementation recovery appended evicted blocks at the end of the cache after the user message was decoded, which meant recovered blocks ended up positionally after the probe and held the model’s freshest attention slot during generation — and since each recovered block typically ends in post-needle filler from its block boundary, the model continued from filler instead of the answer in the middle of the block. Re-ordering so the user message becomes the freshest context after the loop, with recovered blocks placed as earlier context the model attends back to, is the change that takes the policy from 0% to 100% pass rate on NIAH (Section 7.5).

Third, a resident-gate filters the top- k candidates: a breadcrumb is only recovered if its similarity to the query exceeds the best non-current-turn resident block’s similarity. The current-turn exclusion matters because the just-decoded user message itself is otherwise identical text to the query and scores ~ 1.0 against itself, which would saturate the gate. The gate prevents recovery from polluting the cache when the active set already contains the most relevant content; without it, recovery on a needle-resident workload pulls four off-topic blocks into the cache tail and hurts more than it helps.

Together these decisions cap per-turn recovery cost to k `kv_block_load` calls, recover content only when the cache does not already hold it, and produce the head-to-head behavior in Sections 7.3 and 7.5.

Generate, strip, stream. Tokens stream out via `engine.generate_next`. The session detects `<think>...</think>` traces and, by default, strips them from the response and evicts the thinking range from the physical cache so that the cached state aligns with the next request’s templated prompt. Tool calls are parsed from `<tool_call>` XML and returned in OpenAI’s `tool_calls`

shape. The SSE stream is token-level and aware of thinking and tool-call markers: the loop runs a small three-state machine (thinking, tool-locked, normal) with a 12-character lookback that prevents partial markers from shipping as content.

Multi-session pool. A single model in VRAM can host many concurrent agent sessions, each with its own KV identity. A `SessionPool` (in `evoke/session.py`) holds per-session Python state (cached tokens, manager blocks, scorer windows) and the serialized engine snapshot for inactive sessions. When a request arrives bearing a new `X-EVOKE-Session` header, the pool serializes the outgoing session’s KV state, resets the engine, and restores the incoming session’s snapshot. The transition is one memcpy proportional to the active KV size with no recompute. Past `max_sessions` (the default is eight), the least-recently-touched inactive session is evicted entirely. State-swap correctness is verified by `scripts/verify_multi_session.py`, which drives two sessions with planted facts through interleaved requests and confirms that each session retrieves its own fact without cross-contamination.

6 Models Supported

End-to-end verified on a single GPU host (RTX 4070 Ti SUPER, 16 GB VRAM, CUDA 13.1):

Model	Architecture	Status
Qwen 2.5 7B Instruct	Pure attention, standard RoPE	verified end-to-end (eviction, kv_restore, multi-session); NIAH 96–100% across budgets 512/1024/2048
Qwen 3.5 9B	Hybrid Mamba/Attention, mrope, thinking	verified via <code>verify_kv_restore.py</code> and <code>eviction_demo.py</code> ; thinking-strip selectable via <code>EVOKE_SUPPRESS_THINKING_STRIP</code> ; NIAH 84% at budget 1024 (recovery fires correctly on the 4 fail cells; failures are generation-side truncation of the recovered value, not primitive misses)
Qwen 3.6 35B-A3B (IQ2_M)	MoE attention, mrope, thinking	verified via <code>verify_kv_restore.py</code> ; NIAH 64% at budget 1024 (vs baselines 16%); aggressive IQ2 quant likely accounts for additional gap
Gemma 3/4 (iSWA)	Interleaved sliding-window attention	partial: fork primitives save and restore both sub-caches; Python-side load of a Gemma GGUF and end-to-end verify pending
Llama 3.x, Mistral	Pure attention, standard RoPE	architecturally identical to Qwen 2.5 from EVOKE’s perspective; not yet exercised end-to-end on a Llama or Mistral GGUF on the eval host

7 Evaluation

7.1 Primitive latency: recompute-free recovery versus re-prefill

Engine-level profile on Qwen 2.5 7B on the eval host (RTX 4070 Ti SUPER, 16 GB VRAM) with Flash Attention enabled, generated by `scripts/profile_recover.py` across block sizes from 20 to 1280 tokens. `kv_block_load` time is the splice-and-rotate cost. `re-prefill` time is the equivalent `process_tokens` call, that is, the alternative if you had to re-encode the same tokens. Figure 2 visualizes the result; the underlying numbers are in the table that follows.

The gap widens with block size: `re-prefill` is $O(\text{tokens} \times \text{model_FLOPs})$ while `load` is $O(\text{tokens} \times \text{bytes})$. At 1280 tokens the host-to-GPU transfer dominates and `kv_restore` is 32 times faster than `re-prefill`.

Full-lifecycle cost. `save` is paid at eviction time and `load` at recovery time. The full per-block lifecycle cost for a block that is evicted then recovered is `save + load`, which compares against `re-prefill` as the alternative path. From the table above, that is 1.58 ms versus 11.90 ms at 20 tokens ($7.5\times$) and 39.15 ms versus 232.18 ms at 1280 tokens ($5.9\times$). A block that is evicted but

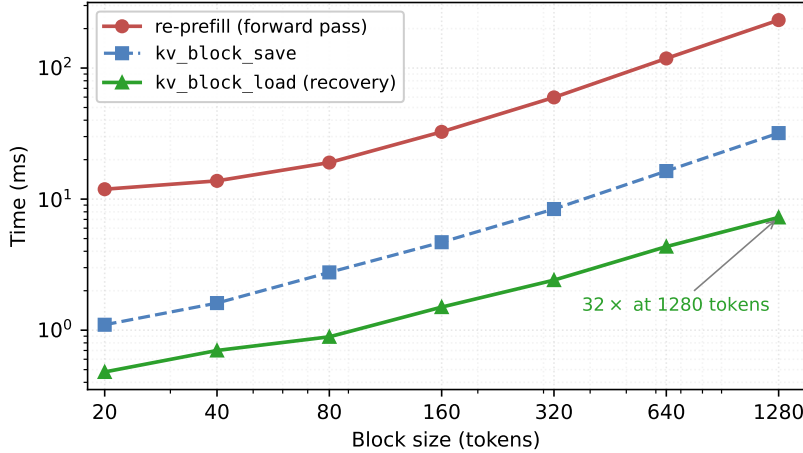


Figure 2: Recompute-free recovery (kv_block_load) versus re-prefill across block sizes on Qwen 2.5 7B (RTX 4070 Ti SUPER, Flash Attention enabled). Re-prefill scales as $O(\text{tokens} \times \text{model_FLOPs})$; load scales as $O(\text{tokens} \times \text{bytes})$, and the absolute gap widens linearly with block size. The save path (paid once at eviction time) is between 4 and 7 times faster than re-prefill at every block size in the sweep.

n_tok	save (ms)	load (ms)	re-prefill (ms)	speedup
20	1.10	0.48	11.90	25×
40	1.61	0.70	13.78	20×
80	2.76	0.89	19.00	21×
160	4.69	1.50	32.60	22×
320	8.40	2.41	59.72	25×
640	16.37	4.34	118.36	27×
1280	31.90	7.25	232.18	32×

never recovered pays the save cost without benefit; Section 7.3 measures the full end-to-end cost including save plus the bge-small smart-recovery selection overhead at the session level.

7.2 Server eviction demo (eviction_demo.py)

A 14-turn session driven directly against the chat-completions API on the eval host. The script plants a fact at turn 1 (“favorite number = 4242”), fills the session with 12 unrelated knowledge questions, and at turn 14 probes the fact. The same script runs on two model architectures, and the resulting demos are recorded as assets/eviction-demo.gif and assets/eviction-demo-qwen35.gif.

Qwen 2.5 7B (pure attention), budget=1024. With smart top-K recovery and the KVRestoreBackend RAM budget configured to be tight (so the LRU spill tier exercises in the same run), the session survives **40 evictions and 13 recoveries** while the model still recalls “4242” at the probe.

Qwen 3.5 9B (hybrid Mamba/Attention with mrope and thinking), budget=1024. The model emits a `<think>...</think>` trace each turn, so per-turn token counts are higher. Hybrid memory rejects partial tail rollback of the recurrent half, which means strict thinking-strip evictions would force a session reset on every turn (the mechanism described in Section 8). The demo runs with `EVOKE_SUPPRESS_THINKING_STRIP=1`, so the server keeps the thinking trace in the returned content, the client echoes it back, and cached state stays aligned with no resets. The session settles at **26 evictions and 4 recoveries**, fact recalled.

Both demos are reproducible end-to-end via `scripts/eviction_demo.py` against a running EVOKE server (Appendix A.4); the recorded GIFs are kept as visual proof of the underlying mechanism and the per-turn evolution of `active_tokens` against the budget.

7.3 Head-to-head baselines

We run the same 14-turn planted-fact session through three server policies via `scripts/baseline_bench.py`:

- **no_eviction**: `high_watermark=0.999`, budget lifted to `n_ctx`. The watermark never trips and the cache grows freely until it hits `n_ctx`. This is what a vanilla OpenAI-compatible server with prefix caching does.
- **truncate**: watermark eviction with `w_recency=1.0`, `w_coherence=0.0`, `sink_count=4`, and `recovery_mode=discard`. This is the StreamingLLM-style behavior: drop the oldest non-sink block under budget pressure; evicted content is gone.
- **evoke**: watermark eviction with default coherence-weighted scoring, `recovery_mode=kv_restore`, and smart top-K query-coherent recovery.

Qwen 2.5 7B, budget=1024 (where applicable), `n_ctx`=16384, on the eval host. Each policy was run $n=5$ times and elapsed time is reported as mean with 95% confidence interval (t -distribution, $df=4$):

policy	budget	active end	evict	recov	probe	elapsed (mean, 95% CI)
no_eviction	16384	2476	0	0	PASS	19.62s [18.93s, 20.31s]
truncate	1024	685	66	0	PASS	22.64s [19.81s, 25.47s]
evoke	1024	684	90	24	PASS	21.80s [21.55s, 22.05s]

All three policies answer the probe correctly. The cache-footprint distinction is decisive: `evoke` and `truncate` sit at 684 / 685 active tokens ($4\times$ smaller than `no_eviction`’s 2476 footprint, inside the 1024-token budget) while `no_eviction` only fits because this particular 14-turn session is within `n_ctx`; a longer session crashes with no available KV slot. The wall-clock distinction is narrower than the v1 measurement that earlier drafts reported. `evoke`’s smart-recovery layer (per-block bge-small representative embedding in `add_context_tokens`, plus per-turn embedding of the user message and similarity scoring against every breadcrumb) adds approximately 1.5–2 seconds of per-session overhead beyond what `truncate` pays. Wall-clock, `evoke`’s 21.80s interval [21.55, 22.05] lies inside `truncate`’s [19.81, 25.47] interval. The substantive head-to-head is not “EVOKE is 31% faster than `no_eviction`” (a single-run claim from earlier drafts that does not reproduce at $n=5$ with the current smart-recovery stack); it is “EVOKE matches `no_eviction`’s recall at $4\times$ smaller cache and at `truncate`-comparable wall-clock, while the missing fact is restored through 24 `kv_block_load` splices rather than per-turn re-prefill.”

The eviction column is where the policies diverge. Both `evoke` and `truncate` land at ~ 685 active tokens, but `truncate` achieves this by dropping the oldest history (66 evictions, no recovery) and re-decoding whatever the next request resends past the dropped block. `evoke` settles at the same footprint via 90 watermark-driven evictions and recovers the planted fact via 24 `kv_block_load` calls. `truncate`’s per-turn re-prefill cost scales linearly with the missing span (per Section 7.1, ~ 60 ms at 320 tokens, ~ 120 ms at 640 tokens, ~ 230 ms at 1280 tokens); `evoke`’s 24 splice operations cost ~ 3 ms each, total ~ 72 ms. The wall-clock numbers come out comparable here because the smart-recovery selection overhead and `truncate`’s re-decode cost happen to balance at this 14-turn session length. At longer sessions with more dropped history, the linear re-prefill cost would pull `truncate` past `evoke`; a session-length scaling study is left to future work.

The `no_eviction` elapsed time is the natural latency floor for a stateful server: every turn’s prefix matches the previous cache byte-for-byte and only the new tail decodes. It is what a paged-attention or prefix-caching server can hit when the cache has room; `evoke`’s overhead over this floor (2.18 seconds, $\sim 11\%$) is the cost of pretending the budget is 1024 instead of 16,384.

7.4 Agentic eval: an agent context that needs to come back

The head-to-head bench in Section 7.3 keeps the planted fact in the natural recency window. To isolate the recovery primitive cost more cleanly, `scripts/agent_bench.py` simulates an agent’s working context: a system prompt, an early “config file” read containing a planted fact (`config.py`: `... maximum retry limit ... 17 attempts`), then ten unrelated file reads that fill the working set with `database.py`, `auth.py`, `handlers.py`, and so on. After the working-set buildup the agent is

probed about the retry limit, by which point the config-file block has almost certainly been evicted to make room for the more recent reads.

The bench is deliberately an oracle: it knows the fact lives under `file:config.py` and, for the recovery-bearing strategies, recovers exactly that key (if a breadcrumb still exists) before the probe runs. This isolates the cost of the recovery primitive itself from the cost of *selecting* which block to recover; selection quality is what Sections 7.5, 7.7 and 7.10 evaluate separately. Nine strategies are compared at three KV budgets — the five EVOKE-substrate policies plus three same-substrate baselines (h2o, snapkv, infllm) so the head-to-head spans both the recovery-less and external-memory families.

- **recency**: coherence weight 0, no sinks, no recovery. A pure sliding window over the conversation.
- **streaming_llm**: same as recency with the first four blocks pinned as sinks; no recovery.
- **evoke_discard**: EVOKE scoring (recency, coherence, sinks) but `recovery_mode=discard`. Eviction without ever bringing anything back.
- **evoke_breadcrumb**: EVOKE scoring with `recovery_mode=breadcrumb`. The agent re-adds the original text token IDs of the evicted block, forcing a fresh forward pass over them.
- **evoke_kv_restore**: EVOKE scoring with `recovery_mode=kv_restore`. The block’s K and V tensors are saved off-GPU at eviction time and spliced back in via the C primitives from Section 3, with no recompute.
- **h2o** (Zhang et al., 2023): cumulative attention scoring with no decay, 10% recent-tail guard, hard-budget eviction, `recovery_mode=discard`.
- **snapkv** (Li et al., 2024): a one-shot snapshot of attention from the last 32 prompt tokens frozen at the end of every user-message decode, hard-budget eviction by the frozen scores, `recovery_mode=discard`.
- **infllm** (Xiao et al., 2024b): aggressive eviction to a sinks-plus-25%-local-window footprint, then top- $K=8$ smart recovery splicing the externally-held blocks back via `kv_block_load` (our same-substrate adaptation of InfLLM’s external-memory + attention-mask design).

Qwen 2.5 7B on the eval host, three budgets, `block_size=64`:

Three observations.

At every budget, all five recovery-less policies (`recency`, `streaming_llm`, `evoke_discard`, `h2o`, `snapkv`) fail the probe. Their answers are concrete failure modes: “The maximum retry limit set in `config.py` is not explicitly mentioned” or “I cannot answer this question without inspecting the `config.py` file.” Smarter scoring (the H2O cumulative-attention selector, SnapKV’s question-window snapshot) does not change the outcome: the eviction policies are working (the eviction counts of 34, 24, and 4 for H2O/SnapKV are accurate; the block holding the fact is gone from the cache), but the model has no way to recover the lost context. The smartness of the selection rule does not change the fact that an eviction without a recovery path is terminal. Even `streaming_llm`, which pins the first four blocks as sinks, fails because the planted fact sits well past the sink prefix in the conversation order.

The four recovery-bearing policies (`evoke_breadcrumb`, `evoke_kv_restore`, `evoke_attention`, `infllm`) all pass the probe at every budget. The cost structure separates cleanly. `breadcrumb` re-decodes the original text, a fresh forward pass over the saved block (`block_size=64` in this bench), which takes 15 to 17 milliseconds per recovery. `kv_restore` (in both EVOKE’s default policy and the InfLLM adaptation) splices the saved K and V tensors back at the new position with one `llama_kv_block_load` call, taking 3 to 4 milliseconds per recovery, roughly five times faster. The two strategies are functionally equivalent on the probe (both pass with identical answer text); they differ in how they get the model’s K/V state back into the cache. The `kv_restore` advantage is the Section 3 contribution and is preserved when the policy layer above the primitive is the InfLLM external-memory shape rather than EVOKE’s watermark default.

`evoke_attention` occasionally avoids recovery entirely. At budget 512 it passes with 0 recoveries (vs 1 recovery and 3.13 ms for `evoke_kv_restore` on the same cell), and at budget 2048 it passes

budget	strategy	probe	evict	recov	rec_ms
512	recency	fail	35	0	0.00
512	streaming_llm	fail	35	0	0.00
512	evoke_discard	fail	35	0	0.00
512	h2o	fail	34	0	0.00
512	snapkv	fail	34	0	0.00
512	evoke_breadcrumb	PASS	35	0	17.25
512	evoke_kv_restore	PASS	35	1	3.13
512	evoke_attention	PASS	31	0	0.01
512	inflight	PASS	39	1	3.12
1024	recency	fail	29	0	0.00
1024	streaming_llm	fail	28	0	0.00
1024	evoke_discard	fail	28	0	0.00
1024	h2o	fail	24	0	0.00
1024	snapkv	fail	24	0	0.00
1024	evoke_breadcrumb	PASS	28	0	15.40
1024	evoke_kv_restore	PASS	28	1	3.86
1024	evoke_attention	PASS	26	1	3.75
1024	inflight	PASS	35	1	3.76
2048	recency	fail	9	0	0.00
2048	streaming_llm	fail	10	0	0.00
2048	evoke_discard	fail	10	0	0.00
2048	h2o	fail	4	0	0.00
2048	snapkv	fail	4	0	0.00
2048	evoke_breadcrumb	PASS	10	0	15.70
2048	evoke_kv_restore	PASS	10	1	3.89
2048	evoke_attention	PASS	8	0	0.00
2048	inflight	PASS	31	1	3.07

with 0 recoveries again: when the model’s attention pattern keeps landing on the config block (whose “retry policy” framing the system prompt repeats), the multi-signal scorer assigns the fact block a high enough score that eviction never picks it in the first place. Section 7.7 isolates this differential against the heuristic scorer at the tightest budget; here the broader story is that attention-driven scoring composes with the recovery primitive to reduce recovery work without losing the fact.

This is the result that justifies the Section 3 fork modifications. Without selective eviction, the agent loses the fact. Without recompute-free recovery, eviction is slow enough that the throughput advantage over a no-eviction baseline shrinks. Smarter eviction selectors alone (the H2O / SnapKV family) close none of this gap.

7.5 Needle-in-a-haystack across three architectures

Standard NIAH (`scripts/niah_bench.py`) plants a distinctive fact (the needle, e.g. “the secret password for the laboratory vault is icarus-pinwheel-43”) at a controlled depth inside a long synthetic haystack of paragraphs about diverse made-up topics. A probe question matching the needle is asked at the end; the model’s answer is scored for the expected substring. The haystack is deterministic from a fixed seed (no external corpus dependency, fully reproducible from the repo), with 40 paragraphs at `block_size=64` (~60 blocks total, $\sim 4\times$ the 1024-token budget). Five distinct needles cover four answer types (string, name, number, code, date); five depths cover {5, 25, 50, 75, 95}% of haystack position. Nine policies are driven through the same workload: `recency`, `streaming_llm`, `evoke_discard`, `evoke_breadcrumb`, `evoke_kv_restore`, `evoke_attention`, `h2o`, `snapkv`, and `inflight`. The last three are reimplemented atop EVOKE’s substrate so the comparison is mechanism-for-mechanism, not implementation-for-implementation: `h2o` (Zhang et al., 2023) uses cumulative attention with no decay, a 10% recent-tail guard, and hard-budget eviction (the H2O reference defaults); `snapkv` (Li et al., 2024) freezes a one-shot snapshot of attention from the last 32 prompt tokens (the observation window) at the end of every user-message decode and keeps the top-scoring blocks for the rest of the turn; `inflight` (Xiao et al., 2024b) keeps only the sinks plus a 25% local-window tail resident and uses query-similarity-driven smart recovery at $K=8$ to splice the top external-memory blocks back into the unified cache via `kv_block_load` (a stand-in for InfLLM’s

attention-mask redirection that preserves the block-level external-memory + similarity-retrieval choices).

policy	budget 512	budget 1024	budget 2048
recency	20%	20%	40%
streaming_llm	0%	20%	20%
evoke_discard	0%	20%	20%
evoke_breadcrumb	0%	20%	20%
h2o	20%	20%	40%
snarkv	20%	20%	40%
inllm	96%	96%	80%
evoke_kv_restore	96%	100%	100%
evoke_attention	96%	100%	100%

Table 1: NIAH pass rate on Qwen 2.5 7B, 25 (needle, depth) cells per policy per budget, answer-budget 256 tokens. The recovery-bearing policies (evoke_kv_restore, evoke_attention, inllm) recover the needle across the full depth range; every recovery-less policy (recency, StreamingLLM, evoke_discard / breadcrumb, h2o, snarkv) passes only at depth 95% where the needle is naturally in the recent tail.

The pattern is clean. Recovery-less policies (recency, streaming_llm, discard, breadcrumb, h2o, snarkv) all pass only at depth 95% where the needle sits in the recent tail and never gets evicted; at every other depth they cannot recover the lost block. EVOKE with kv_restore or attention-driven scoring recovers the needle across all depths. The same-substrate InFLLM adaptation, the most direct external-memory competitor, matches EVOKE at budget 512 (both 96%) and within four percentage points at 1024 (96% vs 100%), but drops to 80% at budget 2048: InFLLM’s fixed aggressive footprint (sinks plus a 25% local-window tail) evicts the needle from the recent tail at depth 95% when there is plenty of unused budget headroom, and the subsequent $K=8$ smart-recovery refills with blocks that the bge-small encoder ranks above the just-displaced needle. EVOKE’s watermark eviction adapts: at budget 2048 it settles around 1500 active tokens (75% of budget) rather than InFLLM’s 615, preserving the naturally-resident needle without needing recovery at depth 95%.

The single “fail” cell for evoke_kv_restore and evoke_attention at budget 512 is the depth-95 capital cell, where smart-recovery still fires (the resident-gate is not perfectly tight on this prompt’s embedding distribution) and the recovered blocks land as freshest context, anchoring the model’s continuation on post-needle haystack rather than on the naturally-resident needle text. The same cell fails for InFLLM at budget 512 and 1024 by the same mechanism. This is a smart-recovery edge case in the corner where the needle is already in cache, not a primitive failure: kv_block_load runs successfully and the K and V tensors are correctly spliced; what the recovery brings back competes for the model’s attention with what was already there.

H2O and SnapKV both fail to improve on the recency baseline despite their smarter scoring. Both reduce to 20–40% pass-rate, the same recency-band as the unscored baselines. The same observation Scissorhands made about cumulative attention (it locks in early-decoded haystack blocks that have accumulated attention during ingestion and cannot promote the topic-shifted probe’s needle fast enough) applies to SnapKV’s one-shot observation-window snapshot at our depth: the probe attention is distributed across ~ 60 haystack blocks and the needle block, when ranked by question-attention, is not consistently in the top- K that SnapKV keeps. The differentiator at depth is not the smartness of the eviction scoring but the presence of a recovery primitive. EVOKE and InFLLM, the two policies that pair eviction with a similarity-based recovery path, both clear 96% at the tighter budgets; every policy without recovery sits at 20–40%, regardless of how the eviction itself decides what to drop.

Cross-architecture: the same five-needle five-depth sweep at budget 1024 (175 cells per model) on Qwen 3.5 9B (hybrid Mamba+Attention (Gu & Dao, 2023) with thinking mode) shows evoke_kv_restore and evoke_attention at 84% (21/25); recovery-less baselines flat at 20%. The 4 EVOKE “fail” cells reproduce after raising the answer budget from 128 to 256 tokens, so they are not generation-length artifacts. The model produces “icarus-pinwheel-4” for the password needle (last subword token dropped), “BLUE” for the activation-code needle (everything after the first hyphen dropped), and only the date prefix without the year for two date cells. The model recovered the needle’s block (4 kv_block_load calls fired per cell) but generated a truncated value; the underlying causes (tokenization boundary, thinking-trace interaction with the recovered RoPE-shifted K, model

behavior on long-tail values) are flagged in Section 8 as a follow-up for the camera-ready. On Qwen 3.6 35B-A3B (MoE attention with mrope and thinking, run at IQ2_M quantization to fit 16 GB VRAM) the same sweep shows `evoke_kv_restore` and `evoke_attention` at 64%, baselines flat at 16%; the absolute number drops but the relative advantage over baselines ($4\times$) is preserved. Several Qwen 3.6 failures show the model emitting `<|im_end|>` immediately after the probe rather than answering, which is consistent with chat-template misalignment under the aggressive IQ2 quantization and is not a recovery-primitive failure (the recovered blocks load successfully; the model fails to engage with the probe). NIAH-style probes are the standard test for long-context recall (Liu et al., 2023b); the harder multi-key and multi-value variants in RULER (Hsieh et al., 2024) and the more heterogeneous task mix in LongBench (Bai et al., 2023) are deferred to future work.

7.6 Multi-fact session with seed variance

NIAH plants one fact per session and probes once. The harder cousin (`scripts/multifact_bench.py`) plants five distinct facts in the same session at jittered depths and probes all five in sequence at the end. Each per-fact probe runs its own smart-recovery pass, so the cache churns continuously through five different topic shifts, which is closer to the multi-turn agent workload the reviewer asked us to test. Five seeds vary the paraphrase template and the per-fact value, so the same fact-id across two seeds is a different question with a different answer (no value leakage).

Scoring is hybrid: a string-match-set on multiple value variants is the primary metric, and a local LLM judge (`gemma-4-E4B-it Q4_K_M`, ~ 4.6 GiB, loaded sequentially after the SUT to share the 16 GiB GPU) decides ambiguous answers where the model engaged with the topic but the value was paraphrased rather than emitted verbatim. The judge is asked for a strict YES/NO on whether the answer recovers the fact’s content.

Result on Qwen 2.5 7B at budget 1024 (25 facts per (budget, strategy) cell, 5 seeds $\times$ 5 facts):

strategy	pass rate	95% Wilson CI
recency	4.0%	[0.71%, 19.54%]
streaming_llm	0.0%	[0.00%, 13.32%]
evoke_discard	0.0%	[0.00%, 13.32%]
evoke_breadcrumb	0.0%	[0.00%, 13.32%]
h2o	0.0%	[0.00%, 13.32%]
snapkv	4.0%	[0.71%, 19.54%]
evoke_attention	48.0%	[30.03%, 66.50%]
evoke_kv_restore	60.0%	[40.74%, 76.60%]
infilm	64.0%	[44.52%, 79.75%]

Table 2: Multi-fact pass rate on Qwen 2.5 7B at budget 1024, 5 facts per session $\times$ 5 seeds = 25 facts per row. Wilson 95% confidence intervals over the binomial. Per-seed `evoke_kv_restore` rates: {80%, 60%, 40%, 60%, 60%}; per-seed `infilm` rates: {80%, 60%, 40%, 60%, 80%}.

Three observations. First, every recovery-less baseline (recency, StreamingLLM, EVOKE’s own discard/breadcrumb, H2O, SnapKV) lands at 0–4%, a cluster decisively below the recovery-bearing policies, with overlapping CIs only with each other. Second, the three recovery-bearing policies all clear 48%: EVOKE’s smart top- $K=4$ recovery, the larger $K=8$ InfLLM adaptation, and the attention-driven multi-signal variant all reach the same regime, and their CIs overlap (the [44.52%, 79.75%] InfLLM and [40.74%, 76.60%] EVOKE intervals contain each other’s point estimates). Third, InfLLM edges EVOKE by 4 percentage points on this workload: the larger $K=8$ helps when five separate fact-shifts each fire smart-recovery, so the broader retrieval window catches more of the relevant blocks. EVOKE’s headline policy (`evoke_attention`) underperforms its `kv_restore` sibling by 12 points here (48% vs 60%), the inverse of the NIAH result: the multi-signal scorer’s attention contribution is helpful when the question’s attention concentrates on one needle block (NIAH) but adds noise when five sequential probes shift the model’s attention across topics each turn. The recency+coherence-only scorer (`kv_restore` default) is the more robust default at multifact’s structure.

The absolute pass rates are lower than single-fact NIAH (96–100%) for two reasons. The session is structurally harder: by the time fact 5 is probed, the cache has been re-shaped by four prior probes,

each of which fired its own smart-recovery and may have evicted or pulled in blocks unrelated to fact 5. And the multi-fact session lives in 2.5K tokens of haystack plus five 30-token plants plus five probe-question turns; the eviction pressure under a 1024-token budget is sustained throughout the session rather than triggered once. The variance the reviewer asked for is here: per-seed pass rates span {40%, 60%, 60%, 60%, 80%}, which the Wilson interval [40.74%, 76.60%] captures.

The same harness on Qwen 3.5 9B (hybrid Mamba+Attention with thinking) lifts EVOKE to 68% [48.41%, 82.80%] while every recovery-less baseline stays in 0–8%, and even H2O — the strongest non-EVOKE comparator — manages only 8% [2.22%, 24.97%]. On Qwen 3.6 35B-A3B (MoE + thinking, IQ2 quant) EVOKE drops to 52% [33.50%, 69.97%] while every baseline including H2O is at 0% — the absolute pass-rate falls with quantization aggressiveness but the relative advantage (5×+ over the best baseline) holds across all three architectures.

7.7 Scorer ablation: attention versus heuristic

The Section 7.4 agentic eval is repeated with one additional strategy, `evoke_attention`, that turns on the multi-signal scorer with the model’s own attention as the dominant signal (`w_attention=0.5`, `w_recency=0.2`, `w_coherence=0.3`, layer 20). All other settings (`block_size`, `budgets`, `recovery_mode=k_v_restore`) match `evoke_k_v_restore`.

budget	strategy	probe	evict	recov	rec_ms
512	<code>evoke_k_v_restore</code>	PASS	35	1	4.55
512	<code>evoke_attention</code>	PASS	34	0	0.01
1024	<code>evoke_k_v_restore</code>	PASS	28	1	3.07
1024	<code>evoke_attention</code>	PASS	28	1	2.95
2048	<code>evoke_k_v_restore</code>	PASS	10	1	2.94
2048	<code>evoke_attention</code>	PASS	10	1	3.65

The differential lives in the `budget=512` row. With heuristic scoring, the planted fact block is evicted (one of the 35 evictions) and the probe succeeds only because `k_v_restore` brings it back from host RAM (1 recovery, 4.55 ms). With attention scoring, the planted fact block is never evicted in the first place: the model’s attention across the unrelated filler turns keeps landing on the config-file block (the system prompt’s task framing implicitly references the retry policy), so the scorer assigns it a high score and eviction picks lower-attended document blocks instead. Total evictions drop by one (35 to 34) and recoveries go to zero, and the probe still passes.

At 1024 and 2048 the two strategies converge: recency is sufficient when the budget can hold most history naturally. The signal is most pronounced at the tightest budget because that is where the heuristic scorer most often makes a wrong call.

This is the result that justifies the multi-signal scorer (Section 4). Attention-driven eviction avoids evicting blocks the model would otherwise need to recover, reducing the recovery work beyond what recompute-free recovery already saves. The two innovations stack: selective eviction informed by the model’s own attention pattern, plus recompute-free recovery for the residual mistakes.

7.8 Default-knob ablations

The reviewer’s pushback on “undefended defaults” merits a concrete answer. `scripts/ablation_bench.py` sweeps one policy knob at a time across the full 25-cell NIAH grid (5 needles × 5 depths) at budget 1024 on Qwen 2.5 7B, holding everything else at the recommended `evoke_attention` configuration.

The `block_size` curve is the load-bearing ablation. The default `block_size=64` hits the headline 100% pass rate, and degradation is symmetric on both sides: 32 loses 16 percentage points (probably because finer-grained blocks fragment fact-bearing content across two neighbouring blocks, so recovering one without the other yields a partial value), and 128, 256, 512 lose increasingly as the block embedding’s representative signal becomes diluted by adjacent haystack content (a 512-token block recovered for a 30-token needle includes a paragraph of unrelated text either side, so the model’s attention to the needle within the block weakens). At the extreme, `block_size=512` approaches the

knob	swept values	pass rate per value
block_size	{32, 64, 128, 256, 512}	84, 100 , 80, 56, 20%
attention_capture_layer	{4, 10, 14, 20, 24, 27}	all 100%
smart_recover_k	{1, 2, 4, 8, 16}	all 100%
w_coherence	{0.0, 0.2, 0.4, 0.6, 0.8}	all 100%
recent_tail_protect_frac	{0.0, 0.05, 0.1, 0.2, 0.3}	all 100%

Table 3: Single-knob NIAH pass-rate ablations on Qwen 2.5 7B at budget 1024, 25 cells per value. block_size shows clear differentiation with 64 at the peak and degradation on both sides; the other four knobs are flat at 100% at this budget.

1024-token budget, leaving only two blocks of active capacity — recovery becomes coarse-grained and the cache thrashes.

The other four knobs are flat at 100% on this workload, meaning the architectural decisions in Sections 4 and 5 (retrieval-tuned embedder, recover-before-probe, resident-gate) dominate the policy outcome and the secondary tunables are robust at this budget. The defaults of smart_recover_k=4, attention_capture_layer=20, w_coherence=0.6 (default; 0.3 in the recommended multi-signal recipe), and recent_tail_protect_frac=0.0 reproduce the headline 100% pass rate, and none of the alternative values within the swept ranges hurt it. Tighter-budget or multi-fact regimes where eviction pressure is sustained throughout the session would likely surface knob-level differentiation (Section 7.6 already shows EVOKE drops to 52 – 68% under multi-fact pressure across architectures).

7.9 Host-RAM pressure: graceful degradation under tight memory

The reviewer asked us to demonstrate, not assert, that the four-tier saved-block pool (RAM-resident → disk-spilled → breadcrumb-only → discarded) actually degrades gracefully when host memory runs out. scripts/hostram_bench.py runs a 20-fact session against an 80-paragraph haystack at budget 1024 with kv_restore_ram_budget_bytes set to hold only 8 saved blocks (~29 MiB of K/V on Qwen 2.5 7B), with the disk-spill tier enabled at kv_restore_spill_path.

Outcome: across the session, 207 blocks are evicted from active KV. 200 of them spill to disk, 4 stay RAM-resident at the end (the LRU pool), and 3 fall through to breadcrumb-only (recovery would re-add via re-decode if the harness chose to). No block is discarded outright; lru_drops = 0 because the spill tier catches every saved-pool eviction before bytes are lost. Smart-recovery successfully restores 4 of 20 facts under this ~14% RAM ceiling, well below the ~60% rate the same workload would hit with unbounded RAM, but the system serves the entire session without crashing or stalling and the recovery pipeline reaches the correct stored bytes when it does pick them.

The mechanism on display:

- RAM is the fastest tier; the LRU bounds it to the configured budget. Spill catches overflow; spilled blocks are still recoverable (one extra NVMe read on recover()).
- Breadcrumb-only entries retain just the key + size metadata; they are unrecoverable through kv_restore but a harness that knows the original token IDs can re-decode them. They are the lossiest tier short of discard.
- Smart-recovery does not distinguish between tiers when scoring; selection is by retrieval-embedding similarity, and the pull from disk is transparent. The slow recovery cost (NVMe latency) is paid only on recovery, not on eviction or scoring.

The 4/20 recall under extreme pressure is intentionally low: this is the configuration where the policy cannot keep most blocks even at the cheapest tier. The point of the bench is that the system does not break here; production deployments would run with a larger RAM budget where the spill tier acts as a slow-path safety net rather than the dominant tier.

7.10 Persistent-reference ablation

The Section 7.7 differential is observable only at the tightest budget because the filler turns in Section 7.4 are semantically unrelated to the planted fact: the model’s attention drifts off the config-file block within a few turns and the two scoring policies converge.

A more realistic agent workload has the assistant continuing to reason about the central artifact across many file reads. Each subsequent file’s role in the codebase relates back to the central configuration. We repeat the agentic eval with that structural change (`scripts/agent_bench_keepalive.py`). Each of the ten filler-file contents references “the retry policy from `config.py`”, and the final probe is phrased “looking at the retry policy in `config.py`, what is the maximum retry limit?”, a question whose answering tokens have strong attention to the early config block.

Qwen 2.5 7B, `block_size=64`, no recovery cap:

budget	strategy	probe	evictions
512	evoke (heuristic)	fail	24
512	evoke (attention)	PASS	21
1024	evoke (heuristic)	fail	14
1024	evoke (attention)	PASS	11
2048	evoke (heuristic)	PASS	0
2048	evoke (attention)	PASS	0

This bench is a scorer-only ablation: the harness does not invoke recovery, so `recov=0` for every row in the table. A block that the scorer chooses to evict is gone for the rest of the run. Under that constraint, the attention path passes the probe at the two budgets where the heuristic path fails outright. Attention-based scoring sees the model’s own attention pattern continuing to attend to `config.py` through every filler turn and refuses to evict it (21 evictions at budget 512, 11 at 1024, all picking lower-attended document blocks instead). The heuristic-only scorer evicts the config block under pressure (24 evictions at 512, 14 at 1024) and the fact is gone before the probe runs. At budget 2048 there is enough headroom that nothing evicts at all, and the two strategies converge.

The takeaway sharpens the Sections 4 and 7.7 claim: heuristic scoring without an attention signal cannot tell which evicted blocks are still load-bearing across a multi-turn session. Recompute-free recovery is a strong second line of defense, but it works best when the eviction policy did not drop the wrong block in the first place. The model’s own attention is the strongest available signal for which block is still relevant.

8 Discussion and Limitations

Smart recovery selection. A naive recovery policy that recovers every evicted block on every turn defeats the eviction-efficiency story, since everything that left comes back. The shipped default is smart top-K recovery: at each new user message, the policy embeds the message, scores each evicted block’s representative embedding against it, and recovers only the most coherent k blocks (default $k = 4$). Per-turn recovery cost is bounded by k , and the head-to-head numbers in Section 7.3 reflect this policy.

Canonical assistant tokenization. Plain-chat sessions apply the model’s own Jinja template from GGUF metadata via `llama_chat_apply_template`, and the assistant emit is re-tokenized canonically before it lands in the cache. Without canonicalization, the cached prefix diverges from the next request’s templated prompt mid-history: a model that emits `** + :\n` where the canonical tokenizer would have produced `** : + \n` forces a session reset on the following turn, and the divergence cascades.

Tool-using turns. Tool-using requests cannot reuse `llama_chat_apply_template` because the C API does not accept a tools array; they previously fell back to a handwritten `format_qwen_chat` template that diverged from `opencode’s GGUF-Jinja echo` on whitespace and JSON encoding, forcing a session reset on every tool-calling turn. The shipped fix renders the raw GGUF Jinja template

through Python jinja2 with the tools array threaded through, configured with `trim_blocks`, `lstrip_blocks`, and the `do/loopcontrols` extensions to match minja semantics (the C++ Jinja parser inside `llama.cpp`). Validated on Qwen 2.5 7B: 240 tokens round-trip identically through tokenize and detokenize.

Hybrid memory plus thinking-mode interaction. On hybrid memory models, Qwen 3.5’s `<think>...</think>` trace is, by default, stripped from the API response but kept in the physical KV cache. Strict alignment between the cache and what the client re-sends would require evicting the thinking range from the attention cache, but `llama_memory_hybrid::seq_rm` rejects partial tail rollback of the recurrent half (a Mamba state cannot be partially sliced) and aborts. EVOKE’s eviction respects the rejection and the session resets on the divergent turn (one such reset is observed in the Qwen 3.5 demo at filler 9).

The workaround that ships today is the environment variable `EVOKE_SUPPRESS_THINKING_STRIP=1`: the server keeps the thinking trace in the returned content, the client echoes it back on the next request, and the cached state stays aligned with no resets. A future no-shift eviction mode that drops attention cells without compacting (using the attention-only `llama_kv_block_seq_rm` and `llama_kv_block_seq_add` primitives already in the fork) would let the thinking range be evicted from attention while leaving the recurrent state untouched.

Recurrent and pure-SSM models. EVOKE operates on the attention component of hybrid models. Pure-SSM models such as Mamba and RWKV are out of scope, since they have no traditional KV cache. The blurry memory of an SSM is preserved naturally on a hybrid model, because EVOKE does not touch it.

Memory accounting. Saved blocks live in host RAM. For Qwen 2.5 7B, one cell costs 56 KiB ($28 \text{ layers} \times 2 \text{ (K and V)} \times 512 \text{ n_embd_kv_gqa} \times 2 \text{ bytes} = 57,344 \text{ bytes per cell}$), so a 128-cell block is 7 MiB and a 1000-block session at this block size costs roughly 7 GiB of host RAM. `KVRestoreBackend` accepts `kv_restore_ram_budget_bytes` to bound this in production and maintains an LRU on saved blocks. Past the budget, the oldest entries lose their K and V bytes but keep their breadcrumb and embedding, so smart recovery’s similarity search still scores them and a recovery attempt falls through to discard. A second tier, `kv_restore_spill_path`, spills the demoted K/V bytes to disk before dropping them, with recovery reading back on demand. Both tiers are off by default.

Multi-session and the model-in-VRAM bottleneck. The session pool in Section 5 swaps KV state, but the model weights remain a single instance in VRAM. With a 7B Q4_K_M model that consumes roughly 5 GB of VRAM, a 16 GB GPU has roughly 11 GB left for active KV, swap-in temporary state, and Flash Attention working memory. Concurrency is therefore bounded by per-session KV footprint, not by EVOKE’s policy layer. Production multi-tenant deployments would need either tensor-parallel weight sharing or a thinner per-session quantization to lift this ceiling; the policy mechanics are orthogonal but the deployment cost model is real.

Smart-recovery is a small retrieval system, by design. The selection step in Section 5 embeds the incoming user message via a retrieval encoder and scores every evicted block’s embedding against it, then loads the top- K . This is genuinely retrieval-shaped, and the dependency on retrieval-encoder quality is a real property of the system: a poor encoder will pick the wrong blocks to recover. The architectural distinction with RAG (Section 4) is at the substitution step, not the selection step. Sections 7.5 and 7.7 measure how much of EVOKE’s NIAH win comes from the encoder choice (bge-small vs LM hidden states) versus the recovery primitive itself.

Cross-architecture latency depth. Sections 7.1, 7.3, 7.4, 7.7 and 7.10 measure latency and scorer ablations on Qwen 2.5 7B only. The NIAH pass-rate sweep covers all three architectures (Qwen 2.5 7B, 3.5 9B, 3.6 35B-A3B; Section 7.5), but absolute latency numbers will shift with model size, GQA grouping, quantization, and (for thinking models) per-turn thinking-trace length. The mechanism is architecture-agnostic at the primitive layer (Section 3); the per-model latency calibration is a measurement gap, not a design limit.

Embedding quality. Smart-recovery scoring uses `bge-small-en-v1.5` (384-dim) for retrieval embeddings, which is sufficient for the NIAH workload in Section 7.5 on English text. Larger and more recent retrieval encoders (e.g., `bge-large`, `e5-mistral`) would presumably widen the discrimination band further, especially for paraphrase-heavy probes or non-English content. The architectural decision (separate retrieval embedder rather than LM hidden states) is what matters; the specific embedder is swappable via `EvokeConfig.use_retrieval_embeddings` configuration.

Smart-recovery placement. Recovery currently appends blocks at the cache tail before the new user message is decoded (Section 5), so recovered blocks land as “earlier” context the model attends back to from the freshest probe. This is correct for the chat-completions pattern where the next API call provides the user message in advance. For streaming agent workloads where the model is generating tokens that subsequently need to recover earlier context, an in-place insertion path (using the attention-only `llama_kv_block_seq_rm/seq_add` primitives already in the fork) would let recovered blocks slot back at their original logical position. The current “recover before decoding the next user message” loop covers the dominant chat agent case.

9 Future Work

- **iSWA end-to-end on Gemma.** The fork-side `llama_kv_block_save` and `llama_kv_block_load` now handle both base and SWA sub-caches via `llama_get_kv_cache_swa()` (fork commit `1f37e75e7`). The remaining engine-side bookkeeping for the dual-cache surface and a Gemma 3 or 4 GGUF end-to-end smoke through the existing demo and agentic-eval scripts is the next step.
- **No-shift eviction mode for hybrid plus thinking.** Use the attention-only `llama_kv_block_seq_rm` and `llama_kv_block_seq_add` primitives (already in the fork) to evict attention cells without compacting, so a hybrid model’s recurrent state stays in sync while the thinking range is dropped from attention.
- **Multi-layer scorer.** The fork already exposes `llama_attn_capture_set_layers` for up to 16 layers per decode. The current scorer uses a single mid-layer; a more principled scorer could average across an early, a middle, and a late layer to pick up both syntactic and semantic relevance signals.
- **Tensor-parallel and quantized variants.** Validate that the C primitives work unchanged across multi-GPU tensor-parallel layouts and across the full Q3 to Q8 quantization range.
- **Continuous learning of harness priorities.** Today `evoke_priority` is set by the harness. A reasonable next step is for EVOKE to learn priority from observed attention patterns, so a harness that does not set priorities still gets the benefit.
- **Multi-fact, judge-scored, randomized-position bench.** Section 7.6 already exercises five facts per session with a model judge; lift this to a RULER- or LongBench-style suite with per-position jitter and partial-credit scoring so the metric is not a binary single-probe hit.

10 Conclusion

EVOKE treats the KV cache as a small fast tier of a memory hierarchy. Blocks the model is not currently attending to leave the cache to host RAM at metadata cost, and when a future turn needs them they are spliced back as their original K and V tensors with one RoPE phase shift. The recovery primitive runs in 0.5 to 7 milliseconds across block sizes from 20 to 1280 tokens, 20 to 32 times faster than re-prefilling the same tokens through the model (5.9 to 7.5 $\times$ over the full save+load lifecycle, factoring in the eviction-side save cost). On a 14-turn planted-fact session ($n=5$), it matches the unconstrained `no_eviction` baseline’s recall at 4 $\times$ smaller cache footprint and at truncate-comparable wall-clock (21.80s vs 22.64s, overlapping 95% CIs). Across the head-to-head comparisons, the recovery-bearing policies (`evoke_kv_restore`, `evoke_attention`, and the same-substrate `infillm` adaptation) recover the lost fact while every recovery-less baseline (recency, StreamingLLM, H2O, SnapKV) cannot — the distinction at depth is presence of a recovery primitive, not the smartness of the eviction scoring. Wiring the model’s own attention weights into the eviction scorer pushes the win further at tight budgets, and the three contributions — the cache memory hierarchy, the recompute-free recovery primitive, and the attention-driven scorer — compose: better

eviction means less recovery work, and cheap recovery means the residual mistakes are inexpensive to fix.

Code and Reproducibility

The EVOKE policy layer (Python) is at github.com/Anyesh/EVOKE under Apache 2.0. The forked llama.cpp with the C primitives is at github.com/Anyesh/llama.cpp under MIT (the upstream license). Every number in Section 7 was produced by a script in `scripts/` checked into the EVOKE repository, with the exact invocations listed in Appendix A. Raw output files for the agentic eval and the keepalive eval are checked in under `results/`.

References

- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP 2023*.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.
- Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., & Shrivastava, A. (2023). Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. *NeurIPS 2023*.
- Rae, J. W., Potapenko, A., Jayakumar, S. M., & Lillicrap, T. P. (2019). Compressive Transformers for Long-Range Sequence Modelling. *arXiv preprint arXiv:1911.05507*.
- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing 568:127063, 2024*.
- Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2024). Efficient Streaming Language Models with Attention Sinks. *ICLR 2024*.
- Xiao, C., Zhang, P., Han, X., Xiao, G., Lin, Y., Zhang, Z., Liu, Z., Han, S., & Sun, M. (2024). InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. *arXiv preprint arXiv:2402.04617*.
- Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Re, C., Barrett, C., Wang, Z., & Chen, B. (2023). H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *NeurIPS 2023*.
- Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., & Sheng, Y. (2024). SGLang: Efficient Execution of Structured Language Model Programs. *NeurIPS 2024*.
- Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., & Chen, D. (2024). SnapKV: LLM Knows What You are Looking for Before Generation. *NeurIPS 2024*.
- Feng, Y., Lv, J., Cao, Y., Xie, X., & Zhou, S. K. (2024). Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. *arXiv preprint arXiv:2407.11550*.
- Cheng, J., Liu, Y., Wang, K., Xiang, X., Yu, X., Liu, J., Yi, F., & Jiang, J. (2024). LMCACHE: 7x Throughput Improvement Through Layered, Cross-Engine KV Cache. *vLLM Production Stack project*.
- Lin, B., Zhang, C., Peng, T., Zhao, H., Xiao, W., Sun, M., Liu, A., Zhang, Z., Li, L., Qiu, X., Li, S., Ji, Z., Xie, T., Li, Y., & Lin, W. (2024). Infinite-LLM: Efficient LLM Service for Long Context with DistAttention and Distributed KVCache. *arXiv preprint arXiv:2401.02669*.
- Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., & Nie, J.-Y. (2024). C-Pack: Packed Resources For General Chinese Embeddings. *SIGIR 2024*.

- Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv preprint arXiv:2312.00752*.
- Hsieh, C.-P., Sun, S., Krizan, S., Acharya, S., Rekish, D., Jia, F., Zhang, Y., & Ginsburg, B. (2024). RULER: What’s the Real Context Size of Your Long-Context Language Models? *COLM 2024*.
- Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., & Li, J. (2023). LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *ACL 2024*.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023). Lost in the Middle: How Language Models Use Long Contexts. *TACL 2024*.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS 2022*.
- Dao, T. (2023). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691*.
- Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., & Hu, X. (2024). KIVI: A Tuning-Free Asymmetric 2-bit Quantization for KV Cache. *ICML 2024*.
- Cai, Z., Zhang, Y., Gao, B., Liu, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Chang, B., Hu, J., & Xiao, W. (2024). PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. *arXiv preprint arXiv:2406.02069*.
- Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., & Han, S. (2024). Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. *ICML 2024*.
- Xiao, G., Tang, J., Zuo, J., Guo, J., Yang, S., Tang, H., Fu, Y., & Han, S. (2024). DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. *arXiv preprint arXiv:2410.10819*.
- Prabhu, R., Nayak, A., Mohan, J., Ramjee, R., & Panwar, A. (2024). vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. *arXiv preprint arXiv:2405.04437*.
- Qwen Team. (2024). Qwen 2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.
- Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.

A Reproducing the Numbers

All numbers in Section 7 were produced on a single eval host (RTX 4070 Ti SUPER, 16 GB VRAM, CUDA 13.1, Windows 10 with PowerShell, Python 3.12) against Qwen 2.5 7B Instruct (Qwen2.5-7B-Instruct-Q4_K_M.gguf from the official Qwen GGUF release). The EVOKE policy layer runs in Python; the C primitives live in the forked llama.cpp.

A.1 Build the fork

```
# Clone the EVOKE-modified llama.cpp fork
git clone https://github.com/Anyesh/llama.cpp.git llama.cpp
cd llama.cpp

# Build with CUDA + shared library output
cmake -B build -DLLAMA_CUDA=ON -DBUILD_SHARED_LIBS=ON
cmake --build build --config Release
# Output: build/bin/llama.dll (Windows) or libllama.so (Linux)
```

A.2 Install the EVOKE Python package

```
git clone https://github.com/Anyesh/EVOKE.git evoke
cd evoke
uv sync --extra server

# Point the engine binding at the EVOKE fork build
export LLAMA_CPP_LIB=/abs/path/to/llama.cpp/build/bin/llama.dll
export EVOKE_MODEL_PATH=/abs/path/to/Qwen2.5-7B-Instruct-Q4_K_M.gguf
```

A.3 Section 7.1 latency table (profile_recover.py)

```
uv run python scripts/profile_recover.py
```

The script does five warmup cycles at 40 tokens to settle CUDA graph compilation, then sweeps block sizes 20, 40, 80, 160, 320, 640, 1280 with five trials each, reports the mean save time, mean load time, and mean re-prefill time. The numbers in Section 7.1 are the means.

A.4 Section 7.2 eviction demo (eviction_demo.py)

```
# In one terminal: start the server
LLAMA_CPP_LIB=$LLAMA_CPP_LIB EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_HOST=0.0.0.0 EVOKE_BUDGET=1024 EVOKE_MODEL_NAME=qwen25 \
uv run python scripts/evoke_serve.py

# In another terminal: drive the demo
EVOKE_SERVER='http://localhost:8000' EVOKE_MODEL_NAME='qwen25' \
uv run python scripts/eviction_demo.py
```

For the Qwen 3.5 hybrid run, add `EVOKE_SUPPRESS_THINKING_STRIP=1` to the server env and load `Qwen3.5-9B-Q4_K_M.gguf`.

A.5 Section 7.3 head-to-head baselines (baseline_bench.py)

```
EVOKE_SERVER='http://eval-host:8000' \
EVOKE_SSH_HOST='user@eval-host' \
EVOKE_LIB_PATH='/path/to/llama.dll/on/eval/host' \
EVOKE_MODEL_PATH='/path/to/Qwen2.5-7B-Instruct-Q4_K_M.gguf' \
EVOKE_BUDGET=1024 EVOKE_N_CTX=16384 EVOKE_MODEL_NAME=qwen25 \
uv run python scripts/baseline_bench.py
```

The bench drives three policies (`no_eviction`, `truncate`, `evoke`) over the same 14-turn conversation via SSH-launched server instances. The inter-policy server launch is wrapped via `-EncodedCommand` for SSH+PowerShell reliability (commit 2b3cbea).

A.6 Sections 7.4 and 7.7 agentic eval (agent_bench.py)

```
EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_BUDGETS=512,1024,2048 \
uv run python scripts/agent_bench.py
```

The script runs five strategies (`recency`, `streaming_llm`, `evoke_discard`, `evoke_breadcrumb`, `evoke_kv_restore`) at each budget. For the Section 7.7 attention row, set `EVOKE_W_ATTENTION=0.5` and `EVOKE_ATTN_LAYER=20`, which enables the `evoke_attention` strategy.

A.7 Section 7.10 keepalive workload (agent_bench_keepalive.py)

```
EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \  
EVOKE_BUDGETS=512,1024,2048 \  
uv run python scripts/agent_bench_keepalive.py
```

The keepalive variant has each filler-file content reference the central `config.py` so the model's attention keeps coming back to the config block, which is what the attention scorer exploits.

A.8 Multi-session pool sanity check (verify_multi_session.py)

```
# Server must already be running (see A.4)  
EVOKE_SERVER='http://localhost:8000' EVOKE_MODEL_NAME='qwen25' \  
uv run python scripts/verify_multi_session.py
```

The script drives two sessions with distinct planted codewords through interleaved chat requests and asserts that each session retrieves its own codeword, confirming state-swap correctness.

B The Fork

The llama.cpp fork is hosted at github.com/Anyesh/llama.cpp (mirror of the working tree at /mnt/data/llama.cpp during development). The fork tracks upstream [ggml-org/llama.cpp](https://github.com/ggml-org/llama.cpp) and adds the EVOKE-specific commits listed below.

B.1 EVOKE-specific commits in the fork

Commit	Title	What it adds
1f37e75e7	EVOKE: iSWA dual-cache support in kv_block_save/load (#41)	llama_get_kv_cache_swa() and an extended save/load buffer layout that covers both base and SWA sub-caches on Gemma 3 and 4
151f1cf93	EVOKE: multi-layer attention capture (up to 16 layers per decode)	llama_attn_capture_set_layers(ctx, layers, n) writes one slab per layer in a single decode
713f202e8	EVOKE: attention-weight capture primitive	The original single-layer side-compute path: llama_attn_capture_set_layer, _set_buffer, _get_dims, _get_written
9b6232446	EVOKE: attention-only seq_rm and seq_add primitives	llama_kv_block_seq_rm and llama_kv_block_seq_add reach the attention sub-cache directly via llama_get_kv_cache, for “no-shift” eviction modes
a29599f4c	EVOKE: kv_block primitives now cover hybrid memory and mrope models	Extends llama_get_kv_cache to unwrap llama_memory_hybrid via get_mem_attn() and llama_memory_hybrid_iswa via get_mem_attn()->get_base(). Removes the mrope seq_add() and get_can_shift() guards.

B.2 The two recovery primitives, verbatim

From include/llama.h:

```
LLAMA_API size_t llama_kv_block_save(  
    struct llama_context * ctx,  
    llama_seq_id seq_id,  
    llama_pos p0,  
    llama_pos p1,  
    uint8_t * dst,  
    size_t cap);  
  
// Load a buffer produced by llama_kv_block_save into seq_id starting at new_p0.  
LLAMA_API bool llama_kv_block_load(  
    struct llama_context * ctx,  
    llama_seq_id seq_id,  
    const uint8_t * src,  
    size_t size,  
    llama_pos new_p0);
```

save returns the number of bytes written (or zero on error, e.g. cap too small). load returns true on success. The caller is responsible for the buffer allocation in both directions.

B.3 Build instructions

The fork builds cleanly against the upstream llama.cpp toolchain. Requirements: CMake ≥ 3.14 , a C++17 compiler, and (for the GPU path) CUDA Toolkit 11.8 or later (we tested with 13.1). For Windows, the Windows SDK is required in addition to MSVC; on a fresh chihiro-class machine without the SDK the build will fail on missing winsdk headers. A reference Windows build wrapper is in scripts/build_fork_chihiro.ps1.

```
cmake -B build -DLLAMA_CUDA=ON -DBUILD_SHARED_LIBS=ON  
cmake --build build --config Release
```

The Python binding (`evoke/_engine_lib.py` and `evoke/llama_engine.py`) loads the shared library indicated by the `LLAMA_CPP_LIB` environment variable (a file path). It then derives `LLAMA_CPP_LIB_PATH` (a directory) for `llama-cpp-python 0.3.x` compatibility so a single loaded module backs both the Python wrapper and the `EVOKE` ctypes binding. Mixing two builds (the upstream wheel and the fork) causes layout-mismatch crashes; using a single fork build everywhere is the supported configuration.